☕ Spark SQL i DataFrames

Zapytania i transformacje danych — bo kawa się nie zbada sama

Python w Zastosowaniach Big Data | Wykład 5

Prowadzący: Pavlo Melnyk

📋 Plan wykładu

1. Szybkie przypomnienie — RDD, architektura Sparka
2. Od RDD do DataFrame — dlaczego structured API wygrywa
3. SparkSession, schematy i typy danych
4. Czytanie danych: CSV, JSON, Parquet
5. `select`, `filter`, `groupBy`, `agg`, `orderBy`
6. Joiny: inner, left, broadcast
7. Spark SQL — surowy SQL na DataFrames
8. DataFrame API vs SQL — pojedynek na przykładach 🥊
9. Pułapki i dobre praktyki

> 🎯 **Po tym wykładzie:** Napiszesz zapytania Spark SQL i DataFrame API, które przeanalizują setki tysięcy transakcji w kilka sekund!

🔄 Szybkie przypomnienie — Wykład 4

Na poprzednim wykładzie poznaliśmy:

- **Architekturę Sparka** — driver, executory, cluster manager
- **RDD** (Resilient Distributed Dataset) — niskopoziomową, rozproszoną kolekcję obiektów
- **Transformacje** (lazy) vs **akcje** (uruchamiają obliczenia)
- Operacje `map`, `filter`, `reduceByKey` na RDD

```
rdd = sc.parallelize([1, 2, 3, 4, 5])
result = rdd.map(lambda x: x * 2).filter(lambda x: x > 4).collect()
```

> 💡 RDD działa — ale trzeba samemu pilnować typów i logiki. Dziś poznamy coś wygodniejszego! ☕

💡 Dlaczego to ważne?

Wyobraź sobie, że jesteś analitykiem danych w sieci kawiarni z **15 lokalami** w różnych miastach. Każdego dnia zbierane są **tysiące transakcji** — kto, co, gdzie i za ile kupił.

Zarząd pyta: "*Który sklep sprzedaje najwięcej latte w piątkowe poranki?*"

Robienie tego na surowych RDD byłoby jak parzenie kawy przez sitko z dziurą wielkości pięści. 😩

DataFrame + Spark SQL = ekspres ciśnieniowy: szybko, precyzyjnie, z pianką na wierzchu. ☕⚡

🤔 RDD vs DataFrame — kiedy co?

Cecha	RDD	DataFrame
Typ danych	Dowolny obiekt Python	Wiersze + kolumny (schema)
Optymalizacja	❌ Brak	✅ Catalyst Optimizer
Wydajność	Wolniejsze	**10–100× szybsze**
SQL support	❌	✅ `spark.sql()`
Czytelność	Wymaga `lambda` na każdym kroku	Deklaratywne — mówisz *co*, nie *jak*

> 🎯 **Zasada kciuka:** Używaj DataFrame / Spark SQL **zawsze**, chyba że musisz operować na surowych, niestrukturalnych obiektach.

🔑 Structured API — dlaczego to rewolucja

DataFrame to **ustrukturyzowana** kolekcja danych — jak tabela w bazie SQL albo arkusz Excela na sterydach 💪

Dzięki temu, że Spark **zna schemat** (nazwy kolumn i typy danych), może:

1. Sprawdzić błędy **przed** uruchomieniem (np. literówkę w nazwie kolumny)
2. Zoptymalizować zapytanie przez **Catalyst Optimizer**
3. Pozwolić Ci pisać **SQL** zamiast tylko `lambda`

```
# RDD — musisz wiedzieć, że element[3] to cena
rdd.map(lambda row: row[3] * row[4])

# DataFrame — kolumny mają nazwy!
df.select(col("cena") * col("ilosc"))
```

🚀 SparkSession — punkt wejścia

Od Sparka 2.0 wszystko zaczyna się od **`SparkSession`**:

```
from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("SiecKawiarni")
    .master("local[*]")     # 👈 użyj wszystkich rdzeni lokalnie
    .getOrCreate()
)

print(spark.version)
```

> 💡 `SparkSession` łączy w sobie dawne `SparkContext`, `SQLContext` i `HiveContext` — jeden obiekt do rządzenia wszystkim.

🧬 Schematy i typy danych

Spark musi wiedzieć, **jakiego typu** są dane w każdej kolumnie:

```
from pyspark.sql.types import (
    StructType, StructField, StringType,
    DoubleType, IntegerType, TimestampType
)

schema = StructType([
    StructField("transakcja_id", IntegerType(), False),
    StructField("sklep", StringType(), False),
    StructField("miasto", StringType(), False),
    StructField("produkt", StringType(), False),
    StructField("kategoria", StringType(), False),
    StructField("cena", DoubleType(), False),
    StructField("ilosc", IntegerType(), False),
    StructField("data_czas", TimestampType(), False),
])
```

> ⚠️ `False` oznacza `nullable=False` — kolumna **nie może** zawierać `null`.

🧬 inferSchema — kiedy Spark zgaduje sam

```
df = spark.read.csv(
    "transakcje.csv",
    header=True,
    inferSchema=True   # 👈 Spark sam wykryje typy
)

df.printSchema()
# root
#  |-- transakcja_id: integer (nullable = true)
#  |-- sklep: string (nullable = true)
#  |-- cena: double (nullable = true)
```

Podejście	Szybkość wczytania	Kontrola typów
`inferSchema=True`	Wolniejsze (dodatkowy skan pliku)	❌ Zgaduje
Jawny `StructType`	Szybsze	✅ Pełna kontrola

> 🎯 W produkcji: **zawsze jawny schemat**. Na warsztatach: `inferSchema` jest OK.

📂 Czytanie danych — CSV, JSON, Parquet

```
# CSV – najpopularniejszy, ale "głupi" format
csv_df = spark.read.csv("dane.csv", header=True, schema=schema)

# JSON – zagnieżdżone struktury
json_df = spark.read.json("dane.json")

# Parquet – kolumnowy format binarny, króluje w Big Data
parquet_df = spark.read.parquet("dane.parquet")
```

Format	Rozmiar pliku	Szybkość odczytu	Schema wbudowana?
CSV	Duży	Wolna	❌
JSON	Duży	Średnia	Częściowo
Parquet	**Mały** (kompresja)	**Bardzo szybka**	✅

> 💡 **Parquet to złoty standard Big Data** — kolumnowy, skompresowany, ze schematem w środku.

🤔 Think About It

Pytanie: Dlaczego format **kolumnowy** (Parquet) jest szybszy niż CSV przy analitycznych zapytaniach typu `SELECT SUM(cena) FROM transakcje`?

📊 Nasz dzisiejszy dataset — Sieć kawiarni ☕

Pracujemy z **~100 000 transakcji** z 15 kawiarni w różnych miastach Polski:

Kolumna	Opis
`data_czas`	Data i godzina transakcji
`sklep`	Nazwa/ID sklepu (kawiarni)
`miasto`	Miasto, w którym znajduje się sklep
`produkt`	Nazwa produktu (np. "Cappuccino")
`kategoria`	`kawa` / `herbata` / `przekąska` / `inne`
`cena`	Cena jednostkowa (PLN)
`ilosc`	Liczba sztuk w transakcji
`metoda_platnosci`	`karta` / `gotówka` / `blik`

🔧 select i filter — podstawy

```
from pyspark.sql.functions import col

# select – wybierz kolumny
transakcje.select("sklep", "miasto", "cena")

# select z transformacją
transakcje.select(
    col("sklep"),
    col("cena") * col("ilosc")       # 👈 wartość transakcji
        .alias("wartosc")
)

# filter / where – synonimy
transakcje.filter(col("kategoria") == "kawa")
transakcje.where(col("cena") > 15.0)
```

> 💡 `filter()` i `where()` robią **dokładnie to samo** — wybierz ten, który brzmi bardziej naturalnie.

🔧 withColumn — dodawanie kolumn

```python
from pyspark.sql.functions import col, when, lit

# Dodajemy kolumnę z wartością transakcji
transakcje2 = transakcje.withColumn(
    "wartosc_transakcji",
    col("cena") * col("ilosc")
)

# Kategoryzacja pory dnia
transakcje3 = transakcje2.withColumn(
    "pora_dnia",
    when(col("godzina") < 12, lit("rano"))
    .when(col("godzina") < 17, lit("popoludnie"))
    .otherwise(lit("wieczor"))
)
```

> ⚠️ `withColumn` **nie modyfikuje** oryginalnego DataFrame — zwraca nowy (immutability)!

📊 groupBy i agg — agregacje

```
from pyspark.sql.functions import count, sum as _sum, avg, round as _round

# Przychód per miasto
przychod_miasto = (
    transakcje
    .withColumn("wartosc", col("cena") * col("ilosc"))
    .groupBy("miasto")
    .agg(
        _sum("wartosc").alias("przychod"),
        count("*").alias("liczba_transakcji"),
        _round(avg("wartosc"), 2).alias("srednia_transakcja")
    )
    .orderBy(col("przychod").desc())
)

przychod_miasto.show()
```

> ⚠️ Importujemy `sum as _sum` — bo `sum` to wbudowana funkcja Pythona!

📊 orderBy — sortowanie wyników

```
from pyspark.sql.functions import desc, asc

# Sortowanie malejąco
przychod_miasto.orderBy(desc("przychod"))

# Sortowanie po wielu kolumnach
transakcje.orderBy(asc("miasto"), desc("wartosc_transakcji"))

# Skrót – znak minus
transakcje.orderBy(col("wartosc_transakcji").desc())
```

> 💡 `orderBy` i `sort` to **synonimy** w PySparku — używaj tego, który wolisz.

🔗 Joiny — łączenie DataFrames

Mamy drugą tabelę: `sklepy` — dane o lokalizacji kawiarni (region, powierzchnia, data otwarcia).

```
# Inner join – tylko pasujące wiersze
result = transakcje.join(sklepy, "sklep")  # 👈 klucz po nazwie kolumny

# Left join – zachowaj wszystkie transakcje, nawet bez matcha
result = transakcje.join(sklepy, "sklep", "left")
```

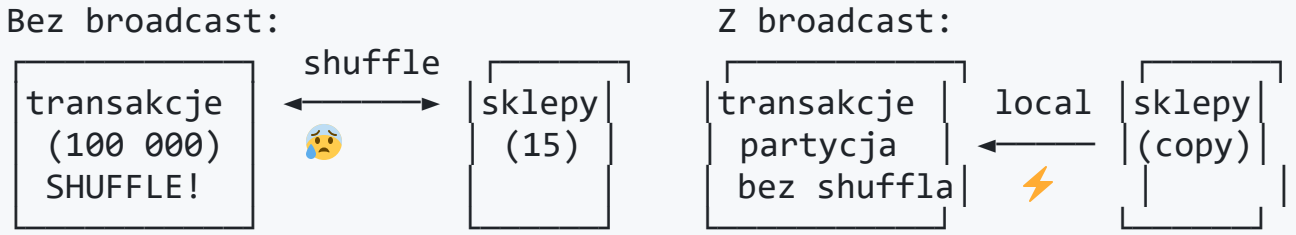
Typ joina	Co dostajemy	Kiedy używać
`inner`	Tylko pasujące wiersze	Domyślny, najczęstszy
`left` / `left_outer`	Wszystko z lewej + pasujące z prawej	Chcemy zachować "główną" tabelę
`full_outer`	Wszystko z obu tabel	Pełny obraz, dużo `null`-i

⚡ Broadcast Join — mała tabela, wielki zysk

`sklepy` ma tylko **15 wierszy** — idealny kandydat na **broadcast join**!

```python
from pyspark.sql.functions import broadcast

# Wymuszamy wysłanie małej tabeli do KAŻDEGO executora
wynik = transakcje.join(
    broadcast(sklepy),   # 👈 kopiuj sklepy wszędzie, zero shuffla!
    "sklep"
)
```



> 🎯 Spark **automatycznie** broadcastuje tabele < 10MB, ale warto wiedzieć, jak to wymusić ręcznie!

💬 Spark SQL — surowy SQL na DataFrames

Wolisz pisać SQL? Spark to umożliwia!

```
# Rejestrujemy DataFrame jako tymczasowy widok
transakcje.createOrReplaceTempView("transakcje")
sklepy.createOrReplaceTempView("sklepy")

# Teraz piszemy SQL jak w bazie danych!
wynik = spark.sql("""
    SELECT s.miasto, SUM(t.cena * t.ilosc) AS przychod
    FROM transakcje t
    JOIN sklepy s ON t.sklep = s.sklep
    GROUP BY s.miasto
    ORDER BY przychod DESC
""")

wynik.show()
```

> 💡 `createOrReplaceTempView` tworzy widok **tylko na czas życia sesji** — nie zapisuje niczego na dysk.

🥊 DataFrame API vs SQL — pojedynek #1

Zadanie: przychód per miasto, posortowany malejąco.

```
# 🐍 DataFrame API
(
    transakcje
    .withColumn("wartosc", col("cena") * col("ilosc"))
    .groupBy("miasto")
    .agg(_sum("wartosc").alias("przychod"))
    .orderBy(col("przychod").desc())
)
```

```
-- 💬 Spark SQL
SELECT miasto, SUM(cena * ilosc) AS przychod
FROM transakcje
GROUP BY miasto
ORDER BY przychod DESC
```

> 🤯 Oba generują **identyczny plan wykonania** — Catalyst optymalizuje je tak samo!

🥊 DataFrame API vs SQL — pojedynek #2

Zadanie: najlepiej sprzedający się produkt w każdym sklepie.

```python
# 🐍 DataFrame API
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number

w = Window.partitionBy("sklep").orderBy(col("przychod").desc())
ranking = sprzedaz_produktow.withColumn("ranga", row_number().over(w))
ranking.filter(col("ranga") == 1)
```

```sql
-- 💬 Spark SQL
SELECT * FROM (
    SELECT *, ROW_NUMBER() OVER (
        PARTITION BY sklep ORDER BY przychod DESC
    ) AS ranga
    FROM sprzedaz_produktow
) WHERE ranga = 1
```

> 🎯 Wybierz to, co **czytelniejsze** dla Ciebie i zespołu — wydajność jest taka sama!

🤔 Think About It

Pytanie: Który zapis jest "lepszy" — DataFrame API czy `spark.sql()`?

A) DataFrame API — zawsze szybszy
B) SQL — zawsze szybszy
C) Nie ma znaczenia — oba przechodzą przez Catalyst Optimizer

⚠️ Common Pitfall #1 — brak `otherwise()`

```
# 🐛 Co się stanie z wierszami, które nie pasują do żadnego warunku?
transakcje.withColumn(
    "segment_cenowy",
    when(col("cena") > 20, "premium")
    .when(col("cena") > 10, "standard")
    # brak .otherwise(...) 🤔
)
```

Wynik: wiersze niepasujące dostają `null` — a `null` potrafi **cicho psuć** dalsze agregacje!

> ✅ **Zawsze** dodawaj `.otherwise(...)` na końcu łańcucha `when()`.

⚠️ Common Pitfall #2 — join bez klucza = eksplozja danych

```
# 🪱 Zapomniałeś podać klucza joina?
transakcje.crossJoin(sklepy)  # 💥 100 000 × 15 = 1 500 000 wierszy!
```

Cross join to iloczyn kartezjański — **każdy z każdym**. Łatwo wygenerować miliony niepotrzebnych wierszy.

> ✅ Zawsze podawaj **jawny warunek** joina (`"sklep"` albo `df1.sklep == df2.sklep`).

⚠️ Common Pitfall #3 — `show()` na dużym DataFrame

```
# 🦎 To wygląda niewinnie, ale...
transakcje.orderBy("wartosc_transakcji").show()  # sortuje WSZYSTKIE 100k wierszy!
```

`show()` domyślnie wyświetla 20 wierszy — ale Spark i tak musi **posortować cały DataFrame**, zanim je pokaże.

> ✅ Filtruj i ograniczaj dane **przed** sortowaniem, jeśli interesuje Cię tylko fragment wyników.

📝 Podsumowanie — kluczowe wnioski

1. ✅ **DataFrame** = RDD + schema + Catalyst Optimizer = szybciej i bezpieczniej
2. ✅ `select`, `filter`, `groupBy`, `agg`, `orderBy` — podstawowy zestaw narzędzi
3. ✅ Joiny (`inner`, `left`, `broadcast`) łączą dane z różnych źródeł
4. ✅ `spark.sql()` + `createOrReplaceTempView` = pełny SQL na DataFrames
5. ✅ DataFrame API i SQL generują **ten sam plan wykonania** — wybierz to, co czytelniejsze

🔮 Co dalej?

Dziś na ćwiczeniach: przeanalizujesz ~100 000 transakcji sieci kawiarni ☕ — przychody, godziny szczytu, ulubione produkty klientów!

Wykład 6 (10.11.2026): Zaawansowany Spark SQL — **window functions** i **optymalizacja zapytań** (Catalyst, `explain()`, caching, unikanie shuffli)

Przed następnym wykładem: upewnij się, że rozumiesz `groupBy` / `agg` — będzie to fundament pod window functions!

❓ Pytania?

Dziękuję za uwagę! 🎉

☕ "*Bez dobrych danych nawet najlepsza kawa nie da Ci trafnych insightów.*"

Zapraszam na ćwiczenia — czas na prawdziwe transakcje z sieci kawiarni!